

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE ASSIGNMENT 3

Name:

Student ID:

Section:

1. Consider a sequence of memory address references given below. In the sequence, each word address is provided in both the decimal and binary formats. Below each address, the relative time at which these references occur is also listed. Memory contents and addresses are shown in the second table.

Data Access Sequence

Time	Address	Decimal	Address	Binary
1	M	28	M	00 01 11 00
2	M	40	M	00 10 10 00
3	M	36	M	00 10 01 00
4	M	16	M	00 01 00 00
5	M	52	M	00 11 01 00
6	H 1	36	H 1	00 10 01 00
7	M	8	M	00 00 10 00
8	M	12	M	00 00 11 00
9	H 2	36	H 2	00 10 01 00
10	M	40	M	00 10 10 00
11	H 3	(36)	H 3	00 10 01 00
12	M	44	M	00 10 11 00
13	H 4	52	H 4	00 11 01 00
14	M	60	M	00 11 11 00
15	M	56	M	00 11 10 00
16	H 5	16	H 5	00 01 00 00
17	M	20	M	00 01 01 00
18	M	12	M	00 00 11 00
19	H 6	36	M	00 10 01 00
20	H 7	52	M	00 11 01 00

Memory

Address	Decimal	Address	Binary	Data
8		00 00 10 00		345
12		00 00 11 00		456
16		00 01 00 00		4
20		00 01 01 00		831
24		00 01 10 00		546
28	✓	00 01 11 00		65
32		00 10 00 00		7
36		00 10 01 00		435
40		00 10 10 00		9
44		00 10 11 00		23
48		00 11 00 00		234
52		00 11 01 00		97
56		00 11 10 00		19
60		00 11 11 00		1000

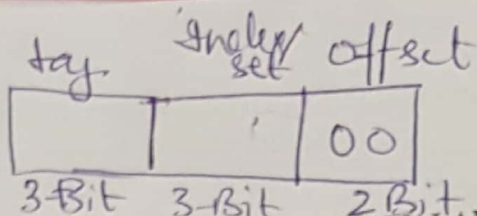
Now consider four different 8-word caches shown below. Assume that each of the four caches was used independently to facilitate memory access for the sequence above. For each cache type, assume that the cache is initially empty. To insert a word into a cache block or set, use the following formulas:

For **direct-mapped cache**: $(\text{memory address} - \text{offset bits}) \text{ modulo } (\# \text{ of blocks in cache})$

For **n-way set associative cache**: $(\text{memory address} - \text{offset bits}) \text{ modulo } (\# \text{ of sets in cache})$

For **fully-associative cache**: a word can be stored in any block

8-Bit address



COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE ASSIGNMENT 3

2. Assume that the least-recently used (LRU) scheme is used where appropriate, where the least recently inserted or accessed element is replaced on conflict. Also, when inserting an element into the cache, if there are multiple empty slots for one index, you should insert the new element into the left-most slot (first available slot).
- a) Use the direct-mapped cache to facilitate memory access for the memory sequence above. You should fill in the binary form of the Tag values. Show the final contents of the cache in the table below, and compute the hit rate and miss rate.

Index	Cache		
	V	Tag	Data
0 000			
1 001		001(56),	435,
2 010		001(40), 000(8), 001(19), 345, 9, 831	
3 011		000(12), 001(46), 000(456), 23, 456	
4 100		000(16),	4
5 101		001(52),	97
6 110		001(56)	19
7 111		000(28), 001(60)	65, 1000 28, 60

Direct-Mapped Cache Summary

Hit Rate:

7/20

Miss Rate:

13/20

- b) Use the 2-way set associative cache to facilitate memory access for the memory sequence above. You should fill in the binary form of the Tag values. Show the final contents of the cache in the table below, and compute the hit rate and miss rate.

Set	2-way Set Associative Cache							
	LR	V	Tag	Data		V	Tag	Data
00 0			0001(16)	4(16),				
01 1			0010(56), 0001(28)	435(56)			0011(52), 0010(26)	435
10 2			0010(40), 0011(56)	97(40), 19(56)			0000(8)	345(8)
11 3			0001(28), 0010(44), 0000(12), 028	65, 23, 44			0000(12), 0011(60)	456(12), 1000(60)

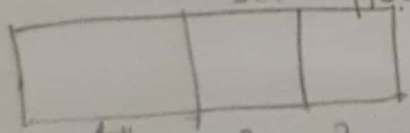
2-way Set Associative Cache Summary

Hit Rate:

6/20

Miss Rate:

14/20 set offset



COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE ASSIGNMENT 3

- c) Use the 4-way set associative cache to facilitate memory access for the memory sequence above. You should fill in the binary form of the Tag values. Show the final contents of the cache in the table below, and compute the hit rate and miss rate.

Set

4-way Set Associative Cache

	V	Tag	Data	V	Tag	Data	V	Tag	Data	V	Tag	Data
0		0010(4)	9		0010(6)	4		0001(8)	345		0011(5)	19
1		0001(28)	68		0010(36)	435		0011(53)	97		0001(2)	456

4-way Set Associative Cache Summary

Hit Rate: $6/20$

Miss Rate: $14/20$

- d) Use the fully-associative cache to facilitate memory access for the memory sequence above. You should fill in the binary form of the Tag values. Show the final contents of the cache in the table below, and compute the hit rate and miss rate.

Fully Associative Cache

V	Tag	Data
	28, 60	
	40 ✓	
	36 ✓✓	
	16 ✓	
	52 ✓✓	
	8, 56	
	12, 20	
	44, 12	

Fully Associative Cache Summary

Hit Rate: $8/20$

Miss Rate: $12/20$

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE ASSIGNMENT 3

3. Fill in the following table for different caches. Assume all caches are direct mapped.

Address Bits	Cache Size	Block Size	Tag Bits	Index Bits	Offset Bits	Bits per Row
16	4KB	4B	4	10	2	37
16	16KB	8B	2	11	3	67
32	8KB	8B	19	10	3	84
32	32KB	16B	17	11	4	146
32	64KB	16B	16	12	4	145
32	512KB	32B	13	14	5	270
64	1024	64B	44	14	6	557
64	2048KB	128B	43	14	7	1068

4. Assume a direct-mapped cache and data access sequence given below where the memory and its contents are shown in the table on the right. Memory is byte addressed and addresses are given in decimal. Assume five-bit addresses and a one-byte data block.

Data Access Sequence

Op.	Address
read	30 M
read	31 M
read	30
read	26 M
read	25 M
read	28 M
read	23 M
read	25
read	28
read	20 M
read	29 M

I

0

1

2

3

Cache

Tag	Data
111 , 101	12 , 8
110 , 111	6 , 4
111 , 110	72 , 43
111 , 101	101 , 13

Memory

Binary	Address	Data
10100	20	5
10101	21	9
10110	22	47
10111	23	13
11000	24	76
11001	25	6
11010	26	43
11011	27	8
11100	28	12
11101	29	4
11110	30	72
11111	31	101

hits =

misses =

hit rate =

miss rate =

3/11

8/11

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE ASSIGNMENT 3

5. Assume a direct-mapped cache and data access sequence given below where the memory and its contents are shown in the table on the right. Memory is byte addressed and addresses are given in decimal. Assume five-bit addresses and a two-byte data block.

Data Access Sequence

Op.	Address
read	30
read	31
read	30
read	26
read	25
read	28
read	23
read	25
read	28
read	20
read	29

I

Cache

Tag	Data 0	Data 1
00	24	25
10	26	27
20	28, 20	29, 21
30	30, 22	31, 23

Memory

Binary	Address	Data
10100	20	5
10101	21	9
10110	22	47
10111	23	13
11000	24	76
11001	25	6
11010	26	43
11011	27	8
11100	28	12
11101	29	4
11110	30	72
11111	31	101

hits =

4

hit rate =

4/10

misses =

6

miss rate =

6/10

6. Assume a 2-way set associative cache and data access sequence given below where the memory and its contents are shown in the table on the right. Memory is byte addressed and addresses are given in decimal. Assume five-bit addresses and a two-byte data block. You should assume that the replacement policy used here is LRU (least recently used).

Data Access Sequence

Op.	Address
lb	30
lb	31
lb	30
lb	26
lb	25
lb	28
lb	23
lb	25
lb	28
lb	20
lb	29

Cache

Tag	Data	Data
0	24, 20	25, 21
1	28	29
	30, 22	31, 23
	26	27

Memory

Binary	Address	Data
10100	20	5
10101	21	9
10110	22	47
10111	23	13
11000	24	76
11001	25	6
11010	26	43
11011	27	8
11100	28	12
11101	29	4
11110	30	72
11111	31	101

hits =

4

hit rate =

4/10

misses =

6

miss rate =

6/10

Consider a sequence of memory access references given below. In the sequence, each address is provided in the decimal format (dec) along with the operation (LDR or STR). Along with each address, the relative time at which these references occur is also listed. Memory contents and addresses are shown in the second table.

Time	Op.	Address (dec)	Data(hex)
1	LDR	M 8	
2	LDR	M 52	
3	LDR	H 12	
4	LDR	M 20	
5	STR	H 16	db db db db
6	LDR	M 44	
7	LDR	H 40	
8	LDR	M 32	
9	STR	M 60	15 05 15 05
10	STR	H 36	20 18 20 18
11	LDR	M 8	
12	LDR	H 12	
13	LDR	M 40	
14	LDR	H 44	
15	LDR	M 48	
16	LDR	H 52	
17	STR	H 40	ca ca ca ca
18	STR	H 44	db db db db
19	STR	M 8	11 22 33 44
20	STR	H 12	55 66 77 88

Address (dec)	Address (bin)	Data (hex)
8	00 00 10 00	aa aa aa aa 11223344
12	00 00 11 00	bb bb bb bb 55667788
16	00 01 00 00	cc cc cc cc dbdbdbdb
20	00 01 01 00	dd dd dd dd
24	00 01 10 00	ee ee ee ee
28	00 01 11 00	ff ff ff ff
32	00 10 00 00	ca ca ca ca
36	00 10 01 00	1A 2B 3C 4D 20182018
40	00 10 10 00	12 34 56 78 caca caca
44	00 10 11 00	78 56 34 12 dbdbdbdb
48	00 11 00 00	ab cd ef 00
52	00 11 01 00	ee 02 04 ee
56	00 11 10 00	00 00 00 00
60	00 11 11 00	11 11 11 11 15051505
64	01 00 00 00	0f 1e 2d 3c

Now consider two different 2-word caches shown on the next page. Assume that each of the two caches are used independently to facilitate memory access for the sequence above. For each cache type, assume that the cache is initially empty.

Assume that the least-recently used (LRU) scheme is used where appropriate, where the least recently inserted or accessed element is replaced on conflict. Also, when inserting an element into the cache, if there are multiple empty slots for one index, you should insert the new element into the first available slot.

For parts (a) and (b) assume the initial memory contents are the ones given above. Update memory in the space above for both parts (try to distinguish between the two if needed.)

Consider a sequence of memory access references given below. In the sequence, each address is provided in the decimal format (dec) along with the operation (LDR or STR). Along with each address, the relative time at which these references occur is also listed. Memory contents and addresses are shown in the second table.

Data Access Sequence				Memory		
Time	Op.	Address (dec)	Data(hex)	Address (dec)	Address (bin)	Data (hex)
1	LDR	M 8 1		8	00 00 10 00	aa aa aa aa 11 22 33 44
2	LDR	M 52 0		12	00 00 11 00	bb bb bb bb 55 66 77 88
3	LDR	H 12 1		16	00 01 00 00	cc cc cc cc d1 b1 b1 b1 b1
4	LDR	M 20 0		20	00 01 01 00	dd dd dd dd
5	STR	H 16 0	db db db db	24	00 01 10 00	ee ee ee ee
6	LDR	M 44		28	00 01 11 00	ff ff ff ff
7	LDR	H 40		32	00 10 00 00	ca ca ca ca
8	LDR	M 32		36	00 10 01 00	1A 2B 3C 4D 20 18 20 18
9	STR	M 60	15 05 15 05	40	00 10 10 00	12 34 56 78 ca ca ca ca
10	STR	H 36	20 18 20 18	44	00 10 11 00	78 56 34 12 d1 b1 b1 b1 b1
11	LDR	M 8		48	00 11 00 00	ab cd ef 00
12	LDR	H 12		52	00 11 01 00	ee 02 04 ee
13	LDR	M 40		56	00 11 10 00	00 00 00 00
14	LDR	H 44		60	00 11 11 00	11 11 11 11 15 05 15 05
15	LDR	M 48		64	01 00 00 00	0f 1e 2d 3c
16	LDR	H 52				
17	STR	H 40	ca ca ca ca			
18	STR	H 44	db db db db			
19	STR	H 8	11 22 33 44			
20	STR	H 12	55 66 77 88			

Now consider two different 2-word caches shown on the next page. Assume that each of the two caches are used independently to facilitate memory access for the sequence above. For each cache type, assume that the cache is initially empty.

Assume that the least-recently used (LRU) scheme is used where appropriate, where the least recently inserted or accessed element is replaced on conflict. Also, when inserting an element into the cache, if there are multiple empty slots for one index, you should insert the new element into the first available slot.

For parts (a) and (b) assume the initial memory contents are the ones given above. Update memory in the space above for both parts (try to distinguish between the two if needed.)

- (a) Use the **direct-mapped** cache shown below with a **block size of two words** to facilitate memory access for the access sequence above. For write hits and misses use the **write back** and **write allocate** policies respectively. You should fill in the binary form of the tag values and hex values for data. When you replace any content in the cache, simply put a cross sign (x) to overwrite a value, and add the new value in the available space. Show the final contents of the memory and cache below, and compute the hit and miss rates.

Miss Rate = $\frac{10}{20} = 0.5$
Hit Rate = $\frac{10}{20} = 0.5$

Cache

D	V	Tag	Data	Data
1	1	001,	cacacaca	1A283C4D, 20182018
1	1	000, 001, 000 001, 000	aaaaaa , 12345678 aaaaaa , 12345678 cacacaca , 11223344	bbbbbb , 78563412 bbbbbb , 78563412 bbbbbb , 78563412
0	1	001, 000, 001	abcedf00 , cacacaca abcedf00 , cacacaca	abcedf00 , cacacaca abcedf00 , cacacaca
1	1	001,	00000000	1811X11, 15051505

- (b) Use the **2-way set associative** cache shown below with a **block size of two words** to facilitate memory access for the access sequence above. For write hits and misses use the **write through** and **write around** policies respectively. You should fill in the binary form of the tag values and hex values for data. When you replace any content in the cache, simply put a cross sign (x) to overwrite a value, and add the new value in the available space. Show the final contents of the memory and cache below, and compute the hit and miss rates.

Cache

D	LRU	Index	V	Tag	Data	Data
0	1	0		0011, 0010	abcedf00 , cacacaca	ee0204ee , 1A283C4D 20182018,
1	1	0		0001, 0011	cccccc , abcedf00 abcedf00 ,	abcedf00 , ee0204ee e
1	1	1		0000, 0011, 0010	aaaaaa , 00000000 12345678 , cacacaca	bbbbbb , 15051505 78563412 , abcedf00
1	1	1		0010, 0000	12345678 , aaaaaa 11223344	78563412 , bbbbbb 55667788

- (c) Which of the above two caches performs better and why?

Miss Rate = $\frac{9}{20} =$
Hit Rate = $\frac{11}{20} =$

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE ASSIGNMENT 3

8. Show the final data and tags of the direct-mapped cache shown below for the given memory contents and data access sequence. As shown, you have 8-bit addresses and 4 bytes of data in each block. You can assume write-through and write-allocate write strategies on write hits and misses respectively.

lw (Load), sw (Store)

Data Access Sequence

Op.	Address	Data
lw	00 01 11 00	M
lw	00 10 10 00	m
sw	00 10 01 00	32 M
lw	00 01 00 00	m
sw	00 11 01 00	642 M
lw	00 10 10 00	H 1
lw	00 01 11 00	H 2
sw	00 10 01 00	124 M
lw	00 10 00 00	M
lw	00 10 10 00	H 3

I

00
01
10
11

Write Through

Direct-Mapped Cache

V	Tag	Data
	000150010	
	0010	
	0001	

Memory

Address	Data
00 00 10 00	435
00 00 11 00	9
00 01 00 00	47
00 01 01 00	13
00 01 10 00	7586
00 01 11 00	86
00 10 00 00	43
00 10 01 00	48
00 10 10 00	673
00 10 11 00	4
00 11 00 00	972
00 11 01 00	1003

hits =

3

misses =

7

hit rate =

3/10

miss rate =

7/10

9. Fill the following, show final cache

Block size = 2

N = 4

Assoc = 2

Memory

10100	20	7
10101	21	3
10110	22	27
10111	23	32
11000	24	101
11001	25	78
11010	26	59
11011	27	24
11100	28	56
11101	29	87
11110	30	36
11111	31	98

Cache

	Address	Data
0	110	24 25
	101	20 21
1	110	26 27

Processor

- Read 24 ✓ M
- Read 25 ✓ H 1
- Read 26 ✓ M
- Read 24 ✓ H 2
- Read 21 ✓ M
- Read 26 ✓ H 3
- Read 24 ✓ H 4
- Read 26 ✓ H 5
- Read 27 ✓ H 6

hits =

6

hit rate =

6/9

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE ASSIGNMENT 3

9. Assume a **direct-mapped cache** of size **8KB**, each block having two 32-bit words. The system is based on 32-bit physical address and uses byte addressing. Find the total number of offsets, index and tag bits.

Off Set Bits =

Index Bits =

Tag Bits =

10. Assume a **4-way set associative** cache of size **16KB**, each block having four 32-bit words. The system is based on 32-bit physical address and uses byte addressing. Find the total number of offsets, index and tag bits.

Off Set Bits =

Index Bits =

Tag Bits =

11. Show final cache where block size = 2, N = 4

memory changed

Memory

Cache

10100	20	7
10101	21	3
10110	22	27
10111	23	32
11000	24	101
11001	25	78
11010	26	59
11011	27	24
11100	28	56
11101	29	87
11110	30	36
11111	31	98

	Address	Data
0	11	24 25
1	11	26, 27, 28, 29 23
2	11	28 27
3	11	30 31

1	Read 30	M	10	Read 30	H
2	Read 31	H	11	Read 31	H
3	Read 30	H	12	Read 30	H
4	Read 26	M	13	Read 26	M
5	Read 25	M	14	Read 25	M
6	Read 28	M	15	Read 28	H
7	Read 23	M	16	Read 23	M
8	Read 25	H	17	Read 25	H
9	Read 28	H	18	Read 28	H

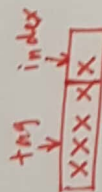
hits = 11

misses = 7

hit rate = $\frac{11}{18}$

miss rate = $\frac{7}{18}$

Show the final data and tags of the two levels of cache, the given memory contents and data access sequence. Consider 5 bit memory addresses and single byte instructions. Assume that the L1 caches are direct mapped and the L2 cache is fully associative, using LRU for replacement. All caches can place a single byte of data in each block. Also assume that addresses greater than 19 contain instructions whereas addresses smaller than 19 contain data.



L1I Cache

Tag	Data
1010 1110	29
1111 1111	5 29 12
1010 1111	17

L1D Cache

Tag	Data
1000	3
0111	86 126
1000	126
0111	

Access Seq.

Op.	Adr.
MM FETCH	20
MM LDRB	15
MM FETCH	30
MM LDRB	16
MM FETCH	31
MM LDRB	15
MM FETCH	20
MM LDRB	17
MM FETCH	28
MM LDRB	15
MM FETCH	30

L1I miss rate = $\frac{6}{6}$
 100%

L1D miss rate = $\frac{4}{5}$
 80%

L2 miss rate = $\frac{7}{10}$
 70%

L2 Cache		Memory	
Tag	Data	Adr.	Data
1010000	5	0110	101
0111111	126	0111	126
1111030	29	1000	3
1000010	3	1001	86
1111112	17	1010	2
		1011	91
		1010	5
		1011	66
		1010	7
		1011	13
		1010	4
		1011	42
		1010	0
		1011	76
		1010	12
		1011	3
		1010	29
		1011	17

Q11:

11. Show final cache where block size = 2, N = 4

memory changes

Memory	Address	Data
10100	20	7
10101	21	3
10110	22	27
10111	23	32
11000	24	101
11001	25	78
11010	26	59
11011	27	24
11100	28	56
11101	29	87
11110	30	36
11111	31	98

	Address	Data
0	11	24 25
1	11	26, 27, 28, 29, 30, 31
2	11	28 27
3	11	30 31

1	Read 30	M	10	Read 30	H
2	Read 31	H	11	Read 31	H
3	Read 30	H	12	Read 30	H
4	Read 26	M	13	Read 26	M
5	Read 25	M	14	Read 25	H
6	Read 28	M	15	Read 28	H
7	Read 23	M	16	Read 23	M
8	Read 25	H	17	Read 25	H
9	Read 28	H	18	Read 28	H

hits = 11

hit rate = $11/18$

misses = 7

miss rate = $7/18$

Q12:

Access Seq.

Op.	Adr.
1	20
2	15
3	30
4	16
5	31
6	15
7	20
8	17
9	28
10	15
11	30

L1I miss rate = $\frac{6}{6}$ 100%

L1D miss rate = $\frac{4}{5}$ 80%

L2 miss rate = $\frac{7}{10}$ 70%

L1I Cache

Tag	Data
1010 1111	29
1111	17

L1D Cache

Tag	Data
1000	3
0111	86, 126
1000	126

L2 Cache

Tag	Data
10100000	5
01111111	126
11110000	29
10000000	3
11111111	17
10001111	86
11110000	12

Memory

Adr.	Data
01110	101
01111	126
10000	3
10001	86
10010	2
10011	91
10100	5
10101	66
10110	7
10111	13
11000	4
11001	42
11010	0
11011	76
11100	12
11101	3
11110	29
11111	17

Q13:

The cache line 10 alternates between holding address 2 and address 6 due to their identical index bits. All accesses result in **cache misses** due to **direct-mapped cache limitations**.

Q14:

1-Way Associativity (Direct-Mapped Cache):

- **Problem:** Strict 1-to-1 mapping causes severe **thrashing**.
- **Cache Misses:** Every access to 222 or 666 evicts the other because they share the same set and there is only 1 block per set.

2-Way Associativity:

- **Improvement:** By allowing 2 blocks per set, addresses 222 and 666 coexist without evictions.
- **Cache Misses:** Only the first access to each unique address causes a miss (cold misses). Subsequent accesses are cache hits.

4-Way Associativity:

- **Best Performance:** The 4 blocks per set allow ample room for both addresses 222 and 666, eliminating any chance of conflict for this sequence.
- **Cache Misses:** Similar to the 2-way case, only the first access to each unique address causes a miss (cold misses).

Q15:

(a) If the hit rate is 90% (h=0.9), then TE=?

$$TE = h \cdot TP + (1-h) \cdot T_{ST_E} = h \cdot T_P + (1-h) \cdot T_{ST_E} = h \cdot TP + (1-h) \cdot TS$$

Substitute the values:

$$TE = 0.9 \cdot 10 + (1-0.9) \cdot 80$$

$$T_E = 0.9 \times 10 + (1 - 0.9) \times 80$$

$$TE = 9 + 0.1 \times 80$$

$$TE = 9 + 8 = 17 \text{ nanoseconds}$$

(b) If the hit rate is 99% (h=0.99), then TE=?

$$TE = 9.9 + 0.01 \cdot 80$$

$$T_E = 9.9 + 0.01 \times 80$$

$$TE = 9.9 + 0.01 \cdot 80$$

$$TE = 9.9 + 0.8 = 10.7 \text{ nanoseconds}$$

$$T_E = 9.9 + 0.8 = 10.7 \text{ nanoseconds}$$

Answer (b): TE=10.7 ns

(c) Suppose a Level-1 Cache ($T_1=4\text{ns}$, $h_1=?$)

The question asks for h_1 , which is not directly solvable here. However, if more context is provided (e.g., TE), we could calculate h_1 .

(d) Suppose a Level-2 Cache ($T_2=10\text{ ns}$, $h_2=?$)

Similar to part (c), h_2 cannot be directly determined from the provided data unless there is more information about TE is given, we can solve for h_2 .

(e) Suppose the main memory has $TS=80\text{ ns}$, $TE=?$

This is redundant because the values for $TS=80\text{ns}$ are already included in parts (a) and (b).

Q16:

.386

Since the Intel 386 relied on **external cache**, the exact parameters varied between motherboard implementations. However, typical configurations included:

Parameter	Typical Value
Associativity	Direct-mapped or 2-way set associative
Number of Blocks	256 to 8192 (depending on cache size and block size)
Number of Sets	Equal to the number of blocks (direct-mapped) or half (2-way set associative)

Q17:

Cache Size: 16 words.

Mapping: Direct-mapped (1 block per set).

Address Format: [3-bit Tag] [4-bit Index].

Number of Blocks in Main Memory: 128.

Q18:

Address Size 6bits

Cache Size 32 words

**Number of
Sets** 2

Blocks per Set 4 (4-way associative)

Block Size 4 words

Tag Size 3 bits

Index Size 1 bit

Offset Size 2 bits

